

# 개인 포트폴리오 - lam

날짜 2026/08/21

## ★ 목적

정상 공정에서 벗어난 웨이퍼를 탐지할 수 있는 최적의 모델 찾기

## 데이터 정보

- 출처 : LAM TCP 9600 금속 에칭 장비에서 실제로 측정된 데이터
- 형식 : calibration(3개의 실험에서 얻은 108개의 정상 웨이퍼 데이터) + test(의도적으로 결함을 발생시킨 21개의 데이터 = 통제된 실험 결함)  
→ 변환) 두 데이터셋을 합친 후, 웨이퍼 id별로 오름차순&시계열 정렬 + result열에 pass/fail 추가
- 특징
  - 데이터 크기(machine data) : 12,829 \* 23 / pass 10,770(83.95%), fail 2,059(16.05%)
  - 열 종류 : 문자열 2개(wafer.id, result) + 숫자열 21개(시간열과 나머지 센서열)
  - 두 단계의 step\_number : 전체 식각 공정이 아닌 특정 스텝(4,5)에 대한 분석
- 숫자형 변수(열) \_공정 단계와 시간 열 제외

### ▼ 상세 설명

#### ▼ RF 계열 : 웨이퍼 바이어스 및 이온 에너지

| 변수                       | 주된 역할                    |
|--------------------------|--------------------------|
| RF Btm Pwr               | 웨이퍼 하부 전극의 바이어스 및 이온 에너지 |
| RF Pwr                   | RF 시스템의 공급·전달 전력         |
| RF Btm Rfl Pwr           | 하부 RF 회로의 반사 전력          |
| RF Tuner / Load          | 하부 RF 매칭 상태              |
| RF Phase Err / Impedance | 하부 RF 회로와 플라즈마의 전기적 상태   |

#### ▼ TCP 계열 : 플라즈마 생성 및 밀도

| 변수                        | 주된 역할                   |
|---------------------------|-------------------------|
| TCP Top Pwr               | 상부 ICP/TCP 플라즈마 소스 전력   |
| TCP Rfl Pwr               | TCP source의 반사 전력       |
| TCP Tuner / Load          | TCP 매칭 네트워크 조정 상태       |
| TCP Phase Err / Impedance | TCP source와 플라즈마의 결합 상태 |

| 그룹화           | 변수 열      | 의미                     | 공정·장비 관점의 설명                                                                                                                                            |
|---------------|-----------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 화학 조성         | BCl3 Flow | BCl <sub>3</sub> 가스 유량 | Boron trichloride의 공급량입니다. AI 식각에서 표면 산화물 제거, 염화물 기반 반응 및 플라즈마 화학 조성에 영향을 줍니다. 일반적으로 장비의 MFC가 설정 유량을 제어하고, 이 값은 실제 또는 제어된 유량을 나타냅니다.                    |
|               | Cl2 Flow  | Cl <sub>2</sub> 가스 유량  | Chlorine 가스 공급량입니다. Al, TiN 등과 반응해 휘발성 염화물 생성에 관여하며, 식각 속도·선택비·식각 profile에 영향을 줍니다. Cl <sub>2</sub> /BCl <sub>3</sub> 비율은 플라즈마의 주요 화학적 상태를 결정합니다.       |
|               | Endpt A   | Endpoint A 검출기 신호      | 식각 endpoint monitor의 광학 방출 신호입니다. 논문에서는 필터 분광기를 이용해 플라즈마 방출광 강도를 측정했습니다. AI 식각 종료와 TiN 식각 구간 등 공정 layer 변화가 신호 profile에 나타납니다.                          |
| 열적/기계적 웨이퍼 상태 | He Press  | He chuck pressure      | 정전척과 웨이퍼 뒷면 사이에 공급되는 헬륨 압력입니다. 웨이퍼와 chuck 사이의 열전달을 유지해 웨이퍼 온도를 제어합니다. He pressure 저하는 wafer backside leak, wafer seating 불량 또는 chucking 문제와 관련될 수 있습니다. |

| 그룹화                   | 변수 열                  | 의미                       | 공정·장비 관점의 설명                                                                                                                                                           |
|-----------------------|-----------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                       | <b>Pressure</b>       | 챔버 압력                    | 식각 챔버 내부의 총 압력입니다. 가스 유량, 배기 속도, throttle valve 개도 및 플라즈마 상태의 결과로 결정됩니다. 압력은 평균 자유 행로, 이온 충돌, 라디칼 수송, 식각 방향성에 직접적인 영향을 줍니다.                                            |
|                       | <b>Vat Valve</b>      | 배기 또는 압력 조절 밸브 개도        | 챔버 배기 라인의 throttle 또는 vat valve 위치입니다. 가스 유입량과 배기량의 균형을 조절하여 챔버 압력을 유지합니다. 밸브 위치 변화는 실제 압력 변화뿐 아니라 펌핑 속도, 배기 컨덕턴스 및 chamber 상태 변화도 반영합니다.                              |
| 이온에너지                 | <b>RF Btm Pwr</b>     | 하부 RF 전력                 | 하부 전극 또는 bottom electrode에 인가되는 RF 전력입니다. 웨이퍼 쪽 이온 에너지와 바이어스 조건에 영향을 주므로, 방향성 식각 및 표면 bombardment와 관련됩니다.                                                              |
| 하부 RF 및 바이어스 상태       | <b>RF Btm Rfl Pwr</b> | 하부 RF 반사 전력              | 하부 RF 회로에서 반사되어 돌아오는 전력입니다. 실제 플라즈마에 전달되지 못한 전력의 크기를 나타냅니다. 반사 전력이 증가하면 임피던스 매칭 불량, 플라즈마 부하 변화, 케이블·전극·매칭 네트워크 이상 등을 의심할 수 있습니다.                                       |
|                       | <b>RF Tuner</b>       | RF tuner 위치 또는 제어량       | RF 전원과 플라즈마 부하 사이의 임피던스를 맞추는 매칭 네트워크의 tuner 위치입니다. 플라즈마 조건이 변하면 최적 매칭점을 유지하기 위해 tuner가 움직입니다. 절대 위치뿐 아니라 시간에 따른 변동도 플라즈마 부하 변화를 반영합니다.                                 |
|                       | <b>RF Load</b>        | RF load 매칭 위치 또는 부하 제어값  | RF 매칭 네트워크의 load 측 조정값입니다. RF generator가 플라즈마에 전력을 효율적으로 전달하도록 부하 임피던스를 조정하는 변수입니다. <b>RF Tuner</b> 와 함께 플라즈마 부하 및 매칭 상태를 나타냅니다.                                       |
|                       | <b>RF Phase Err</b>   | RF 위상 오차                 | RF 전압과 전류의 위상 관계가 목표 매칭 상태에서 얼마나 벗어났는지 나타냅니다. 위상 오차가 커지면 전력이 플라즈마에 효율적으로 전달되지 않거나 반사 전력이 증가할 수 있습니다.                                                                   |
|                       | <b>RF Pwr</b>         | RF 전력                    | RF 시스템에서 공급 또는 전달되는 전력입니다. 장비에 따라 generator forward power, delivered power 또는 제어 전력을 의미할 수 있습니다. 플라즈마 밀도, 전자 에너지, 해리율 및 라디칼 생성량에 영향을 줍니다.                              |
|                       | <b>RF Impedance</b>   | RF 임피던스                  | RF 회로와 플라즈마가 보이는 전기적 임피던스입니다. 플라즈마의 전도도, 전자 밀도, 가스 조성, 압력 및 sheath 상태 변화가 반영될 수 있습니다. RFM 계열 데이터와 관련된 대표적인 플라즈마 상태 지표입니다.                                              |
| 플라즈마 밀도               | <b>TCP Top Pwr</b>    | TCP 상부 전력                | 상부 TCP/ICP 플라즈마 소스에 공급되는 전력입니다. 주로 플라즈마 밀도와 전자·라디칼 생성량을 결정합니다. 일반적으로 하부 RF power보다 <b>플라즈마 생성 및 밀도 제어</b> 측면의 역할이 큼니다.                                                 |
| 상부 TCP/ICP 플라즈마 소스 상태 | <b>TCP Tuner</b>      | TCP 매칭 tuner 위치          | TCP(Transformer Coupled Plasma) 소스의 매칭 네트워크 tuner 위치입니다. 상부 ICP 소스와 플라즈마 사이의 임피던스 매칭을 유지하기 위한 제어값입니다. 플라즈마 밀도와 소스 부하가 변하면 tuner 위치도 변할 수 있습니다.                         |
|                       | <b>TCP Phase Err</b>  | TCP 위상 오차                | TCP 소스의 전압과 전류 사이 위상이 목표값에서 벗어난 정도입니다. TCP source의 매칭 상태, 플라즈마 부하, 방전 안정성을 감시하는 데 사용됩니다.                                                                               |
|                       | <b>TCP Impedance</b>  | TCP 소스 임피던스              | TCP source가 플라즈마에 대해 보이는 전기적 임피던스입니다. 가스 조성, 압력, 플라즈마 밀도 및 chamber condition 변화에 반응합니다. TCP source와 plasma 간 결합 상태를 나타내는 값입니다.                                         |
|                       | <b>TCP Rfl Pwr</b>    | TCP 반사 전력                | TCP source에서 매칭되지 못하고 반사되는 전력입니다. TCP 매칭 상태, 플라즈마 부하 변화, 소스 또는 매칭 네트워크 이상을 나타낼 수 있습니다. 값이 갑자기 증가하면 플라즈마 ignition, source matching 또는 chamber 상태 문제를 의심할 수 있습니다.        |
|                       | <b>TCP Load</b>       | TCP load 매칭 위치 또는 부하 조정값 | TCP 매칭 네트워크의 load 측 조정값입니다. TCP generator가 플라즈마에 전력을 전달하도록 부하 조건을 조정합니다. <b>TCP Tuner</b> , <b>TCP Phase Err</b> , <b>TCP Impedance</b> 와 함께 TCP source의 매칭 상태를 설명합니다. |

|                                                                                                                                 |                          |
|---------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| <b>BCl3 Flow</b> , <b>Cl2 Flow</b>                                                                                              | 식각 가스 유량                 |
| <b>Pressure</b> , <b>He Press</b>                                                                                               | 챔버 압력, 웨이퍼 후면 냉각용 헬륨 압력  |
| <b>RF Pwr</b> , <b>RF Btm Pwr</b> , <b>RF Btm Rfl Pwr</b> , <b>RF Tuner</b> , <b>RF Load</b> , <b>RF PPhase Err</b> , <b>RF</b> | 플라즈마를 발생시키는 RF 파워 및 반사파워 |

|                                                                             |                                             |
|-----------------------------------------------------------------------------|---------------------------------------------|
| Impedance                                                                   |                                             |
| TCP Tuner, TCP Impedance, TCP Top Pwr, TCP Phase Err, TCP Rfl Pwr, TCP Load | TCP(Transformer Coupled Plasma) 소스 관련 튜닝 변수 |
| Endpt A                                                                     | 식각 종료 시점을 감지하는 광학 엔드포인트 신호                  |
| Vat Valve                                                                   | throttle(vat) valve)의 압력 or 밸브 개도           |

## 논문

### 1. 공정

- Al stack etch : TiN/Al - 0.5%Cu/TiN/Oxide
- 단계 : 가스 유량 및 압력 안정화 → 압력 안정화 → 플라즈마 점화 → AI 식각(4) → TiN 및 산화막 overetch(5) → 챔버 벤트
- 품질 지표 : Al 배선의 linewidth 변화, 웨이퍼 내 식각 균일도, 산화막 손실량

### 2. 사용 데이터

| 센서(데이터셋) 종류      | 담고 있는 정보                 | 역할                        |
|------------------|--------------------------|---------------------------|
| Machine-state 센서 | 가스 유량, 압력, RF 전력, 임피던스 등 | 공정 장비 상태를 직접적으로 반영        |
| RFM 센서           | RF 전압, 전류, 위상 관계         | 플라즈마의 전기적/화학적 상태에 민감      |
| OES 센서           | 플라즈마 방출광 분광 데이터          | 플라즈마 내 화학종 및 식각 부산물 정보 제공 |
| 사용 분석법           | PCA, MPCA, TLD, PARAFAC  |                           |

### 3. 연구 내용

- 어떤 센서 조합과 데이터 처리 방법이 식각 공정의 미세한 이상을 가장 잘 검출하는가?
- 정상 상태이더라도 공정 드리프트에 의해 불량 발생
  - 클린 사이클 동안 챔버 벽에 식각 부산물/잔류물 축적
  - 상류 공정 변화에 따른 웨이퍼와 원재료 차이
  - 센서 자체의 장기 드리프트
  - 정기 유지보수 후 공정 평균값이 갑자기 이동
 → 모델이 너무 넓은 정상 영역을 학습하면 작은 fault가 정상 변동에 묻혀 발견하기 어렵게 됨
- Local 모델이 Global 모델보다 fault detection sensitivity가 높음
  - Global 모델은 장기간의 정상적인 공정 drift까지 정상 변동으로 포함 → PCA 모델의 허용 영역이 넓어지고, 작은 fault가 정상 범위 안에 들어가게 됨
- Local 모델은 특정 시점의 공정 상태를 기준으로 하므로 작은 이상도 더 쉽게 검출
- FDC 시스템에서 중요하게 다뤄야 할 운영 전략
  - 최근 데이터 기반의 모델 업데이트
  - 클린 사이클 또는 유지보수 이후 별도 모델 사용
  - 공정 상태별 local model 구성
  - 무조건 장기간 데이터를 하나의 모델에 넣지 않기
- 전처리 방법별 모델 성능에 주는 영향

| 전처리 방법                                         | 장점                                                                                                                                         | 단점                                                                                                                                                       |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 평균값으로 축약<br>(각 웨이퍼의 전체 시간 이력을 평균 해 하나의 벡터로 만들) | <ul style="list-style-type: none"> <li>• 센서 노이즈 감소</li> <li>• Machine-state 데이터의 감도 향상</li> <li>• PCA 적용이 간단함</li> <li>• 계산량 감소</li> </ul> | <ul style="list-style-type: none"> <li>• 시간에 따른 profile 정보 손실</li> <li>• trajectory shape 변화 검출 불가</li> <li>• 특정 공정 단계에서만 발생하는 fault가 묻힐 수 있음</li> </ul> |
| Raw data 사용                                    |                                                                                                                                            |                                                                                                                                                          |
| 웨이퍼별 평균 중심화                                    |                                                                                                                                            |                                                                                                                                                          |
| 시간축 정렬                                         |                                                                                                                                            |                                                                                                                                                          |

#### 4. 반도체 공정 관점에서의 해석

- Fault와 Drift를 구분해야 함
- 센서별로 역할이 다른 걸 인지해야 함

### 분석 과정

#### ▼ 16-2 : 전처리 - 결측치/상수열 확인 + 범주형/연속형 구분

- 결측치/상수열 : 없음
- RF Btm Rfl Pwr(2), TCP Rfl Pwr(7) → 정수형으로만 출력하는 데이터인듯. 각 이온에너지와 플라즈마 밀도에 큰 영향을 주므로 삭제하지 않음
- 다만 반사 전력은 챔버로 들어가지 못하고 다시 반사돼서 발생기 쪽으로 들어오는 전력이라 -는 불가능.  
-1이라 적힌거 노이즈로 보고 0으로 바꿔놓음 (8,858번째 행, 13321/38.7656 (wafer.id/time))

#### ▼ 프롬프트

@data/lam\_etch\_modify.csv 에서 문자열과 Time열을 뺀 모든 숫자형 열에 대해 조건에 맞게 하나의 표로 정리해줘.

표가 너무 길어지면 5개만 보여주고, 펼쳐보기 등을 눌러서 전체로 볼 수 있게 해줘.

1. 센서이름
  2. 값의 범위 (최소~최대)
  3. 서로 다른 값이 몇 가지인지
- 확실하지 않은 건 확실하지 않다고 적어줘.

채팅에만 쓰지 말고, 지금 노트북 가장 아래에 새 코드 칸을 만들어 실행해줘.

#### ▼ 코드

```
import pandas as pd
from IPython.display import display, HTML

df = pd.read_csv('../data/lam_etch_modify.csv')

exclude_cols = ['wafer_id', 'result', 'Time']
numeric_cols = [c for c in df.select_dtypes(include='number').columns if c not in exclude_cols]

summary = pd.DataFrame({
    '센서이름': numeric_cols,
    '최소값': [df[c].min() for c in numeric_cols],
    '최대값': [df[c].max() for c in numeric_cols],
    '고유값 개수': [df[c].nunique() for c in numeric_cols],
})
summary['값의 범위 (최소~최대)'] = summary['최소값'].astype(str) + ' ~ ' + summary['최대값'].astype(str)
summary = summary[['센서이름', '값의 범위 (최소~최대)', '고유값 개수']]

print('문자열 열(wafer_id, result), Time 열을 뺀 숫자형 열 개수:', len(numeric_cols))
print()
print('상위 5개')
display(summary.head(5))

html = summary.to_html(index=False)
display(HTML(f'<details><summary>전체 {len(summary)}개 펼쳐보기</summary>{html}</details>'))
```

```
ls>'))

print()
print('참고(확실하지 않음): 고유값 개수가 아주 적은 열(예: RF Btm Rfl Pwr, TCP Rfl Pwr)은')
print('연속형 센서값이라기보다 사실상 범주형/이진값에 가까워 보이는데,')
print('이게 장비 스펙상 원래 그런 값인지 계속 해상도 한계인지는 이 데이터만으로는 확실하지 않  
다.')
```

#### ▼ 출력 결과

| 센서이름           | 값의 범위(최소~최대)                           | 고유값 개수 |
|----------------|----------------------------------------|--------|
| Step Number    | 4.0 ~ 5.0                              | 2      |
| BCI3 Flow      | 724.4693877551019 ~ 776.4693877551019  | 15     |
| CI2 Flow       | 748.0 ~ 757.0                          | 23     |
| RF Btm Pwr     | 124.0 ~ 142.0                          | 52     |
| RF Btm Rfl Pwr | 0.0 ~ 1.0                              | 2      |
| Endpt A        | 272.0 ~ 3464.0                         | 1855   |
| He Press       | 98.12061224489796 ~ 103.12061224489796 | 10     |
| Pressure       | 942.05179028133 ~ 1410.3295238095238   | 293    |
| RF Tuner       | 9311.0 ~ 9805.0                        | 312    |
| RF Load        | 8811.0 ~ 9321.0                        | 392    |
| RF Phase Err   | -3556.0 ~ 2392.0                       | 3760   |
| RF Pwr         | 18.0 ~ 33.0                            | 16     |
| RF Impedance   | 15584.0 ~ 17629.0                      | 1478   |
| TCP Tuner      | 18724.0 ~ 20680.0                      | 825    |
| TCP Phase Err  | -1655.0 ~ 3084.0                       | 1544   |
| TCP Impedance  | 14878.0 ~ 18124.0                      | 796    |
| TCP Top Pwr    | 334.0 ~ 366.0                          | 149    |
| TCP Rfl Pwr    | -1.0 ~ 8.0                             | 7      |
| TCP Load       | 26794.0 ~ 28640.0                      | 788    |
| Vat Valve      | 44.0 ~ 56.0                            | 13     |

- 참고(확실하지 않음): 고유값 개수가 아주 적은 열(예: RF Btm Rfl Pwr, TCP Rfl Pwr)은 연속형 센서값이라기보다 사실상 범주형/이진값에 가까워 보이는데, 이게 장비 스펙상 원래 그런 값인지 계속 해상도 한계인지는 이 데이터만으로는 확실하지 않다.

#### ▼ 16-4 : 두 개의 csv(step4,5로 나눔)에서 각 열의 기초 통계량 확인

- Step number(2) → step별로 나눠서 데이터 분석 = Step4 (6,025행) + Step5 (6,804행) = 원본(12,829행) 동일

#### ▼ 프롬프트

```
@day06/project_lam/result/lam_etch_step4.csv , @day06/project_lam/result/lam_etch_s  
tep5.csv 각 파일에 대해 모든 열에 대한 네 가지를 계산해서  
열 하나가 한 줄이 되는 표를 만들어줘. 각 step별로 표를 만들어야 하니까 총 2개가 나와야 해.
```

1. 열 이름
2. 빈칸 비율 (%)
3. 서로 다른 값의 개수
4. 표준편차
5. 최솟값, 최댓값

만든 표를 sensor\_diagnosis 라는 이름으로 두고,  
빈칸 비율이 높은 순으로 정렬해서 위에서 10줄만 보여줘.

그리고 이 표가 전부 몇 줄인지도 알려줘.

채팅에만 쓰지 말고, @day06/project\_lam/project\_lam.ipynb 가장 아래에 새 코드 칸을 만들어 실행해줘.  
지어내지 말고 실제 실행값만 보여줘.

#### ▼ 코드

```
import numpy as np
from IPython.display import display

def make_sensor_diagnosis(df, label):
    rows = []
    for col in df.columns:
        s = df[col]
        missing_pct = s.isna().mean() * 100
        nunique = s.nunique()
        if pd.api.types.is_numeric_dtype(s):
            std = s.std()
            vmin = s.min()
            vmax = s.max()
        else:
            std = np.nan
            vmin = s.min()
            vmax = s.max()
        rows.append({
            '열 이름': col,
            '빈칸 비율(%)': round(missing_pct, 4),
            '고유값 개수': nunique,
            '표준편차': std,
            '최솟값': vmin,
            '최댓값': vmax,
        })
    result = pd.DataFrame(rows).sort_values('빈칸 비율(%)', ascending=False).reset_index(drop=True)
    print(f'[{label}] sensor_diagnosis 전체 줄 수: {len(result)}')
    display(result.head(10))
    return result

step4 = pd.read_csv('result/lam_etch_step4.csv')
step5 = pd.read_csv('result/lam_etch_step5.csv')

print('=== lam_etch_step4.csv ===')
sensor_diagnosis_step4 = make_sensor_diagnosis(step4, 'lam_etch_step4')

print()
print('=== lam_etch_step5.csv ===')
sensor_diagnosis_step5 = make_sensor_diagnosis(step5, 'lam_etch_step5')
```

#### ▼ 출력 결과

[lam\_etch\_step4] sensor\_diagnosis 전체 줄 수: 22

|   | 열 이름     | 빈칸 비율(%) | 고유값 개수 | 표준편차      | 최솟값       | 최댓값       |
|---|----------|----------|--------|-----------|-----------|-----------|
| 0 | wafer_id | 0.0      | 129    | NaN       | l2901.txm | l3343.txm |
| 1 | Time     | 0.0      | 3586   | 14.054670 | 8.66797   | 65.559    |

|   | 열 이름           | 빈칸 비율(%) | 고유값 개수 | 표준편차       | 최솟값        | 최댓값         |
|---|----------------|----------|--------|------------|------------|-------------|
| 2 | BCI3 Flow      | 0.0      | 11     | 2.394470   | 724.469388 | 753.0       |
| 3 | CI2 Flow       | 0.0      | 23     | 0.784381   | 748.0      | 757.0       |
| 4 | RF Btm Pwr     | 0.0      | 46     | 1.818129   | 124.0      | 142.0       |
| 5 | RF Btm Rfl Pwr | 0.0      | 2      | 0.107923   | 0.0        | 1.0         |
| 6 | Endpt A        | 0.0      | 1420   | 845.151163 | 296.0      | 3464.0      |
| 7 | He Press       | 0.0      | 10     | 0.752731   | 98.120612  | 103.120612  |
| 8 | Pressure       | 0.0      | 249    | 30.834546  | 942.05179  | 1410.329524 |
| 9 | RF Tuner       | 0.0      | 250    | 40.903540  | 9311.0     | 9805.0      |

[lam\_etch\_step5] sensor\_diagnosis 전체 줄 수: 22

|   | 열 이름           | 빈칸 비율(%) | 고유값 개수 | 표준편차       | 최솟값       | 최댓값         |
|---|----------------|----------|--------|------------|-----------|-------------|
| 0 | wafer_id       | 0.0      | 128    | NaN        | I2901.txm | I3343.txm   |
| 1 | Time           | 0.0      | 4828   | 15.980762  | 54.5879   | 127.259     |
| 2 | BCI3 Flow      | 0.0      | 12     | 2.095481   | 750.0     | 776.469388  |
| 3 | CI2 Flow       | 0.0      | 20     | 0.712598   | 750.0     | 757.0       |
| 4 | RF Btm Pwr     | 0.0      | 31     | 1.353895   | 130.0     | 137.0       |
| 5 | RF Btm Rfl Pwr | 0.0      | 2      | 0.112996   | 0.0       | 1.0         |
| 6 | Endpt A        | 0.0      | 1094   | 461.372696 | 272.0     | 2007.0      |
| 7 | He Press       | 0.0      | 10     | 0.728308   | 98.120612 | 103.120612  |
| 8 | Pressure       | 0.0      | 193    | 23.171996  | 973.05179 | 1393.329524 |
| 9 | RF Tuner       | 0.0      | 278    | 47.787279  | 9314.0    | 9781.0      |

#### ▼ 18 : 시험/학습용 데이터 나누고 기준모델과 로지스틱 회귀 모델 학습 (+실습7 참고)

##### ▼ 프롬프트

@day06/project\_lam/result/lam\_etch\_step4.csv 을 쓸 거야. @day06/project\_lam/project\_lam.ipynb 에 이어서 코드 칸을 만들어줘.

아래 순서 그대로 해줘.

1. 파일을 불러온다 - 경로는 [result/lam\_etch\_step4.csv]
2. 숫자로 된 열만 입력 후보로 삼는다. 이 중 Time은 입력에서 제외한다. wafer\_id도 모델 입력에는 넣지 않지만, 5번에서 웨이퍼 단위로 나눌 때 써야 하니 데이터프레임에서 완전히 지우지는 말고 입력 열 목록에서만 빼준다
3. result는 답이므로 입력에서 뺀다
4. result 열이 "Fail"이면 1, 아니면 0인 label 열을 만든다
5. 학습용과 시험용을 8:2로 나눈다 - 단, 행이 아니라 wafer\_id 단위로 먼저 나눈다. 웨이퍼 목록 (중복 없이)과 그 웨이퍼의 label을 가지고 웨이퍼를 8:2로 나누고 (양쪽 불량 웨이퍼 비율이 같도록), 그다음 그 웨이퍼에 속한 행 전체를 통째로 학습용/시험용에 배정한다. 같은 웨이퍼의 행이 학습용과 시험용에 동시에 들어가면 안 된다
6. 기준 모델 점수를 먼저 켜 - 학습하지 않고 시험용 전부 0이라 답했을 때의 정확도
7. 표준화를 한 뒤 로지스틱 회귀 모델 하나를 학습시킨다 - 표준화 기준 (평균·표준편차)은 학습용에서만 구하고, 시험용에는 그 기준을 적용만 한다 (fit은 학습용에만)
8. 시험용으로 아래 일곱 줄을 출력한다 기준 모델 정확도 / 내 모델 정확도 / 지목한 건수 / 그중 진짜 / 시험용의 실제 건수 / 정밀도 / 재현율

지켜야 할 것

- 코드 칸은 2~3개로 나누고, 줄마다 한국어 주석을 붙여줘
- 시험용은 학습에 쓰지 마 (스케일링 fit도 포함)
- 학습용·시험용은 wafer\_id 기준으로 나눠서, 같은 웨이퍼의 행이 양쪽에 동시에 들어가지 않게 해줘
- 결과를 지어내지 말고 실제로 실행해서 나온 값만 보여줘

▼ 코드

```
# 1. 파일을 불러온다
df = pd.read_csv('result/lam_etch_step4.csv')

# 2. 숫자로 된 열만 입력 후보로 삼는다
numeric_cols = df.select_dtypes(include='number').columns.tolist()
feature_cols = [c for c in numeric_cols if c != 'Time'] # Time은 입력에서 제외
feature_cols = [c for c in feature_cols if c != 'wafer_id'] # wafer_id는 문자열이라 이미 빠져있지만 명시적으로 한 번 더 제외 (5번에서 그룹 나눌 때 df에는 그대로 남겨둠)

# 3. result는 답이므로 입력에서 뺀다 (result는 문자열이라 numeric_cols에 애초에 없음, 확인 차 한 번 더 제외)
feature_cols = [c for c in feature_cols if c != 'result']

# 4. result가 "Fail"이면 1, 아니면 0인 label 열을 만든다
df['label'] = (df['result'] == 'Fail').astype(int)

print('입력으로 쓸 열 개수:', len(feature_cols))
print(feature_cols)

# 5. 학습용과 시험용을 wafer_id 단위로 8:2로 나눈다
from sklearn.model_selection import train_test_split

# 웨이퍼 목록(중복 없이)과 그 웨이퍼의 label을 만든다
wafer_label = df.groupby('wafer_id')['label'].first()

# 웨이퍼를 8:2로 나눈다 (양쪽 불량 웨이퍼 비율이 같도록 stratify)
train_wafers, test_wafers = train_test_split(
    wafer_label.index,
    test_size=0.2,
    random_state=42,
    stratify=wafer_label.values,
)

# 나뉜 웨이퍼에 속한 행 전체를 학습용/시험용으로 통째로 배정한다
train_df = df[df['wafer_id'].isin(train_wafers)]
test_df = df[df['wafer_id'].isin(test_wafers)]

# 같은 웨이퍼가 학습용과 시험용에 동시에 들어가지 않았는지 확인
overlap = set(train_df['wafer_id']) & set(test_df['wafer_id'])

print('학습용 웨이퍼 수:', len(train_wafers), '/ 시험용 웨이퍼 수:', len(test_wafers))
print('학습용 웨이퍼 중 불량 비율:', round(wafer_label.loc[train_wafers].mean(), 4))
print('시험용 웨이퍼 중 불량 비율:', round(wafer_label.loc[test_wafers].mean(), 4))
print('학습용 행 수:', len(train_df), '/ 시험용 행 수:', len(test_df))
print('학습용/시험용에 겹치는 웨이퍼 수(0이어야 정상):', len(overlap))

# 입력(X)과 정답(y)을 나눈다
X_train = train_df[feature_cols]
y_train = train_df['label']
X_test = test_df[feature_cols]
y_test = test_df['label']
```

```
# 6. 기준 모델 점수를 먼저 잰다 - 시험용 전부 0(양품)이라 답했을 때의 정확도
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score

baseline_pred = pd.Series(0, index=y_test.index)
baseline_acc = accuracy_score(y_test, baseline_pred)

# 7. 표준화를 한 뒤 로지스틱 회귀 모델을 학습시킨다
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # 표준화 기준은 학습용에서만 구한다(fit)
X_test_scaled = scaler.transform(X_test) # 시험용에는 구해둔 기준을 적용만 한다(transform)

model = LogisticRegression()
model.fit(X_train_scaled, y_train) # 학습은 학습용으로만
y_pred = model.predict(X_test_scaled) # 예측은 시험용으로만

# 8. 시험용 기준으로 일곱 줄을 출력한다
model_acc = accuracy_score(y_test, y_pred)
지목한_건수 = int((y_pred == 1).sum())
그중_진짜 = int(((y_pred == 1) & (y_test.values == 1)).sum())
시험용_실제_건수 = int((y_test == 1).sum())
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)

print('기준 모델 정확도:', round(baseline_acc, 4))
print('내 모델 정확도:', round(model_acc, 4))
print('지목한 건수:', 지목한_건수)
print('그중 진짜:', 그중_진짜)
print('시험용의 실제 건수:', 시험용_실제_건수)
print('정밀도:', round(precision, 4))
print('재현율:', round(recall, 4))
```

#### ▼ 출력 결과

```
입력으로 쓸 열 개수: 19
['BCL3 Flow', 'CL2 Flow', 'RF Btm Pwr', 'RF Btm Rfl Pwr', 'Endpt A', 'He Press', 'P
ressure', 'RF Tuner', 'RF Load', 'RF Phase Err', 'RF Pwr', 'RF Impedance', 'TCP Tun
er', 'TCP Phase Err', 'TCP Impedance', 'TCP Top Pwr', 'TCP Rfl Pwr', 'TCP Load', 'V
at Valve']
```

```
학습용 웨이퍼 수: 103 / 시험용 웨이퍼 수: 26
학습용 웨이퍼 중 불량 비율: 0.165
시험용 웨이퍼 중 불량 비율: 0.1538
학습용 행 수: 4791 / 시험용 행 수: 1234
학습용/시험용에 겹치는 웨이퍼 수(0이어야 정상): 0
```

```
기준 모델 정확도: 0.8444
내 모델 정확도: 0.8436
지목한 건수: 1
그중 진짜: 0
시험용의 실제 건수: 192
```

정밀도 : 0.0  
재현율 : 0.0

/코드

### 💡 과정 요약

1. 원본 데이터 : 12,829 \* 23 / pass 10,770(83.95%), fail 2,059(16.05%)
  2. step별로 나눈 데이터 : step4 (6025,22), step5(6804,22) → step number 열 하나 빼서 개수 줄음
    - step number 열 하나 빼서 개수 줄음
    - step4의 (I3321.txm/38.7656 = wafer.id/time)행의 TCP Rf1 Pwr 값 0으로 수정. -1은 나올 수 없으므로 노이즈라고 판단함
  3. 기준모델 / 로지스틱 회귀 모델 학습 - step4
    -
-